

Lista de Código: Módulos (import / export)

Turma: LionsDev

Tópicos: `export default`, exports nomeados (`export`), `import`, caminhos relativos (`./` e extensão `.js`), e `"type": "module"` no `package.json`.

Nesta lista você separa o código em arquivos e os conecta com `import` / `export`. Crie uma pasta para a lista com um `package.json` contendo `"type": "module"` e crie os arquivos conforme cada exercício pedir.

Parte 0: Exercícios de fixação

Itens curtos para praticar exportação e importação entre arquivos pequenos.

1. Crie o `package.json` com `"type": "module"`. Sem essa configuração, o Node não aceita `import` / `export` em arquivos `.js`.
2. No arquivo `soma.js`, escreva a função `somar(a, b)` e faça `export default somar`.
3. No `index.js`, faça `import somar from './soma.js'` e imprima `somar(2, 3)`.
4. No `saudacao.js`, exporte por default uma função que retorna `"Olá!"`.
5. No `matematica.js`, use `export nomeado` em duas funções: `dobro` e `triplo`.
6. No `index.js`, importe as duas funções: `import { dobro, triplo } from './matematica.js'`.
7. No `constantes.js`, declare `const PI = 3.14` e, na linha seguinte, faça `export default PI`. (O `export default` não aceita declaração com `const` na mesma linha.)
8. Importe `PI` no `index.js` e imprima a área de um círculo de raio 2.
9. Importe o export default de `soma.js` com outro nome (ex.: `import minhaFuncao from './soma.js'`) e confirme que funciona.
10. Declare o objeto `const config = { versao: 1 }`, exporte por default e importe no `index.js`.

Parte 1: Complete o código

1. Exportando por default

Complete o arquivo `calculadora.js`.

```
// calculadora.js
function calcular(a, b) {
  return a + b;
}

// TODO: exporte 'calcular' como export default
```

```
// index.js
import calcular from "./calculadora.js";
console.log(calcular(4, 6)); // 10
```

2. Exports nomeados

Complete o arquivo `strings.js` exportando as duas funções.

```
// strings.js
export function maiuscula(txt) {
  return txt.toUpperCase();
}

// TODO: exporte também 'minusculta' que retorna txt em minúsculo
```

```
// index.js
import { maiuscula, minusculta } from "./strings.js";
console.log(maiuscula("oi")); // OI
console.log(minusculta("OI")); // oi
```

Parte 2: Ache o bug

3. Import quebrado

Este import dá erro no Node. Aponte os dois problemas.

```
import somar from "soma"; // BUG (dois erros aqui)
console.log(somar(1, 2));
```

4. Default vs nomeado

O arquivo exporta com `export default`, mas o import usa chaves. Conserte.

```
// util.js
export default function formatar(v) { return "R$ " + v; }

// index.js
import { formatar } from "./util.js"; // BUG
console.log(formatar(50));
```

Parte 3: Prever o resultado

5. Vai rodar ou dar erro?

Para cada caso, diga se funciona ou dá erro e por quê.

```
(a) import somar from "./soma.js";           // arquivo usa export default
(b) import somar from "./soma";              // sem extensão .js
(c) import { somar } from "./soma.js";       // arquivo usa export default
(d) import qualquerNome from "./soma.js";    // arquivo usa export default
```

Parte 4: Escreva do zero

6. Mini Projeto Modular (Desafio)

Monte um projeto com 4 arquivos separados:

- `operacoes.js`: exports nomeados de `somar`, `subtrair`, `multiplicar`, `dividir`.
- `formatador.js`: `export default` de `formatarReal(valor)` que retorna `"R$ " + valor`.
- `mensagens.js`: `export default` de um objeto `{ bemVindo: "Bem-vindo!", tchau: "Até logo!" }`.
- `index.js`: importa tudo, calcula `somar(10, 5)`, formata o resultado com `formatarReal` e imprime junto da mensagem de boas-vindas.

Saída esperada:
Bem-vindo!
Resultado: R\$ 15

Dica: para importar um arquivo do próprio projeto, o caminho precisa começar com `./` (ou `../`) e terminar com `.js`. Sem o `./`, o Node procura o pacote em `node_modules`; sem o `.js`, o Node não encontra o arquivo e gera o erro `ERR_MODULE_NOT_FOUND`. O `export default` é importado sem chaves, e os exports nomeados são importados entre `{ }`.

LionsDev • Professor Nicolas Cardoso Motta

Lista de Código: Módulos - Módulo 05